

Comparative Analysis of NMPC and Fuzzy PID Controllers for Trajectory Tracking in Omni-Drive Robots: Design, Simulation, and Performance Evaluation

Love Panta

Electronics and Computer Engineering, Pulchowk Campus, Lalitpur,
44600, Bagmati, Nepal.

Contributing authors: 075bei016.love@pcampus.edu.np;

Abstract

Trajectory tracking for an Omni-drive robot presents a challenging task that demands an efficient controller design. This paper introduces a self-optimizing controller, Type-1 fuzzyPID, which leverages dynamic and static system response analysis to overcome the limitations of manual tuning. To account for system uncertainties, an Interval Type-2 fuzzyPID controller is also developed. Both controllers are designed using Matlab/Simulink and tested through trajectory tracking simulations in the **CoppeliaSim** environment. Additionally, a non-linear model predictive controller (NMPC) is proposed and compared against the fuzzyPID controllers. The impact of tunable parameters on NMPC's tracking accuracy is thoroughly examined. We also present plots of the step-response characteristics and noise rejection experiments for each controller. Simulation results validate the precision and effectiveness of NMPC over fuzzyPID controllers while trading computational complexity. Access to code and simulation environment is available in the following link: <https://github.com/love481/Omni-drive-robot-Simulation.git>.

Keywords: Omni-drive robot, fuzzyPID controller, NMPC, Center of Gravity(COG), Path-tracking, Step-response, Noise rejection

1 Introduction

In recent years, the Omni-directional mobile robot has garnered attention and interest from various research communities due to its unique drive mode, robust maneuver control, and diverse applications across different fields. In comparison to various driving techniques, Omni-based robots offer controlled motion in all directions, allowing them to track user-defined paths or optimal trajectories with low computation costs [1, 2].

The development of an optimal path for a robot to navigate in both static and dynamic environments has been a persistent concern for researchers. Several path-generating algorithms, such as Grassfire, Dijkstra, A^* , and RRT, have been extensively used in mobile robotics to provide optimal solutions [3]. Among these algorithms, A^* is commonly employed for 2D plane navigation, utilizing a heuristic approach to generate the desired trajectory. However, the paths generated by A^* often consist of straight segments and sharp angular turns, which may be undesirable for navigating robots. To address this, path-smoothing techniques are applied to soften breakpoints and enhance parametric continuity [4].

Conventional PID controllers [5] have been widely employed in various robotics application for efficient speed control [6] and waypoint tracking [7]. However, optimizing these controllers for desirable output characteristics can be challenging, and they may not be suitable for a wide range of operating conditions [6, 8]. Consequently, an alternative and effective approach to controller design is utilizing fuzzy logic. Fuzzy logic controllers leverage sets of rules derived from the responses of various static and dynamic systems to adjust tuning parameters based on fuzzy input variables [9–12]. These controllers exhibit good system performance, transient response, and disturbance rejection capabilities. Moreover, they have been successfully applied to various non-linear plants that are challenging to model due to a lack of sufficient parameters [13, 14].

In the context of autonomous path tracking, a fuzzy-based controller has been proposed that leverages the geometric properties of the path ahead for a differential drive robot [15]. This controller considers the curvature and distance to the next bend relative to the robot’s current location, thereby mimicking real driving behavior to determine the desired cruise speed. Similarly, T1-Fuzzy PID controllers have been developed using 25 fuzzy rules for the automatic tuning of PID gains in differential drive robots and quadcopters [7, 16]. The tuned gains are then used to compute the desired velocity, guiding the robot along the reference trajectory and minimizing tracking errors, which demonstrated superior performance compared to classical PID controllers. A Particle Swarm Optimization (PSO)-optimized fuzzy controller has been introduced that directly outputs joint torques for a 2-DOF planar robot [17]. In this approach, PSO is employed to optimize the antecedent and consequent parameters of the Mamdani-based fuzzy logic system. For omni-drive robots, studies have successfully evaluated the performance of fuzzy logic-based path planners [18–20]. These approaches continuously update control signals derived from the fuzzy logic controller based on deviation errors, guiding the robot to the desired waypoints.

Recently, significant research has focused on developing effective Type-2 Fuzzy Logic Controllers (FLCs) to model various sources of uncertainties in nonlinear systems [12, 21, 22]. A Type-2 Fuzzy Logic Controller has been presented for modeling uncertainties in measurements during the tracking of mobile objects in robotic soccer games [23]. This controller uses angle error and change in angle as inputs to the fuzzy system, outputting speeds and directions to control the motion of the agents. Similarly, a novel application of GA-optimized IT2-FPD+I controllers has been explored for 5-DOF redundant robot manipulators to track desired joint trajectories [24]. Their experiments demonstrated robustness in terms of disturbance rejection, model uncertainties, and noise injections, outperforming previous T1-FPD and conventional PID controllers. A new method for building an optimal structure of a PID-type IT2 FLC system has been proposed for controlling joint actions in delta robots [25]. This approach uses the Non-dominated Sorting Genetic Algorithm (NSGA) to tune the controller's scaling factors by formulating the problem as a multi-objective optimization task. In another study, Social Spider Optimization (SSO) has been utilized for parameter tuning [26]. Additionally, the effects of Footprint of Uncertainty (FOU) parameters on generating the control surface for a Single Input IT2-Fuzzy PID (SI-IT2-FPID) controller have been explained, with performance demonstrations for trajectory tracking of UAVs [27].

With the advancement of powerful computing devices, optimization-based control techniques have become integral in various autonomous mobile industries [28]. Model Predictive Control (MPC) is one such method widely used for path tracking of mobile robots, as it takes into consideration various constraints for optimal control inputs [29–33]. In comparison with traditional PID methods, MPC offers higher accuracy, smoother control inputs, and increased resistance to external disturbances. Papers such as [30, 34] also highlight the superior performance of MPC over PID controllers, especially when considering kinematic constraints in mobile robots. Considering this, different variants of MPC-based path tracking controllers have been evaluated in recent years, based on the nature of the prediction model's nonlinearity and the representation of output state quantities [35, 36]. In 2007, the paper by Conceicao *et al.* [37] proposed non-linear MPC based on error kinematics models for Omni-drive robots for path following, which was particularly challenging due to hardware constraints at that time. To address these challenges, linear MPC was introduced, which linearized the robot model to simplify computation operations. In a subsequent paper by Wang *et al.* [38], the authors designed linear MPC and demonstrated its effectiveness for point stabilization and trajectory tracking. Similarly, Kanjanawanishkul *et al.* [31] presented and applied linear error MPC in real-time for Omni-drive robots, considering system and input constraints to trade for stability. Furthermore, Grüne *et al.* [39] introduced various extensions of NMPC for time-varying references with or without terminal constraints. Despite advancements in trajectory tracking, few approaches have been applied to Omni-directional robots, leaving a gap in proposing Nonlinear Model Predictive Control (NMPC) and assessing its performance against other control methods.

We utilize CoppeliaSim as the simulation platform to conduct our experiments. We design the physical model of a four-wheel omni-drive robot in SolidWorks and

import the URDF format into CoppeliaSim. Subsequently, it is interfaced with Matlab/Simulink via an external API to enable control. Physical parameters such as frictional force and air resistance are maintained at default values in the simulation environment. In summary, our paper comprises three main sections. The initial two sections are dedicated to modeling the kinematics of the robot and subsequently addressing path planning. Following this, we introduce an Omni-drive controller, employing two distinct methods: **fuzzy logic with the Mamdani model and Non-linear Model Predictive Control (NMPC)**. We proceed to assess and compare the performance of these approaches in guiding the robot along the reference trajectory generated by the path planning algorithms. The design of the Omni-drive controller involves formulating a non-linear predictive model, treated as a cost minimization problem. The objective is to determine optimal control inputs that adhere to bounded input constraints, subsequently applied to the robot kinematics. This ensures the robot effectively follows the desired path outlined by the path planning algorithms.

2 Kinematics model

To derive the kinematics model of an Omni-drive robot, it is essential to be aware of the geometric configuration of each wheel with respect to the robot's Center of Gravity (COG). This knowledge is crucial for understanding how the global velocity of the robot is distributed to each wheel, assuming the absence of roller skidding. The kinematics model establishes the relationship between wheel angular velocity and the robot's global velocity, and vice versa. In this context, the global coordinate are aligned with the local coordinate frame of the robot. We extend the approach taken by [40] to derive the kinematic of 4-wheel Omni-drive robot.

The global velocity of robot is written as $(\dot{x}, \dot{y}, \dot{\theta})$ and angular velocity of each wheel is denoted by $(\dot{\phi}_1, \dot{\phi}_2, \dot{\phi}_3, \dot{\phi}_4)$ which are at an angle of $\frac{\pi}{4}, \frac{3\pi}{4}, \frac{5\pi}{4}, \frac{7\pi}{4}$ respectively. Robot body radius and wheel radius are also taken as R and r . Then the translation velocity of each wheel hub v_i can be formed as the combination of pure translation and pure rotational part of robot [40]. Thus, Fig. 1 shows each wheel has a velocity equal to the expression given as,

$$v_i = -\sin(\theta + \alpha_i)\dot{x} + \cos(\theta + \alpha_i)\dot{y} + R\dot{\theta} \quad (1)$$

where, $\theta + \alpha_i$ is the offset angle of each wheel w.r.t COG. θ is $\frac{\pi}{4}$ in case of a four-wheel omni robot as shown in Fig. 2 and α_i is $\frac{\pi}{2}(i - 1)$ for the wheel $i=1, \dots, 4$ respectively. Now, the kinematics model of the robot can be represented in matrix form using equation (1) as,

$$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \\ \dot{\phi}_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} -\sin(\theta + \alpha_1) & \cos(\theta + \alpha_1) & R \\ -\sin(\theta + \alpha_2) & \cos(\theta + \alpha_2) & R \\ -\sin(\theta + \alpha_3) & \cos(\theta + \alpha_3) & R \\ -\sin(\theta + \alpha_4) & \cos(\theta + \alpha_4) & R \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} \quad (2)$$

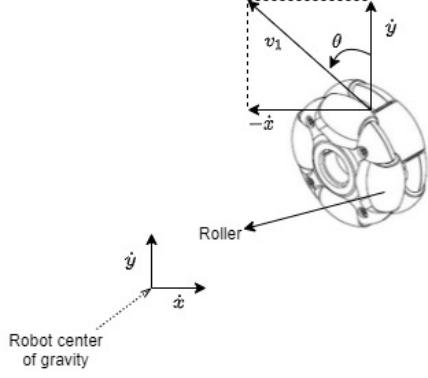


Fig. 1 Component division of translational velocity of omni wheel

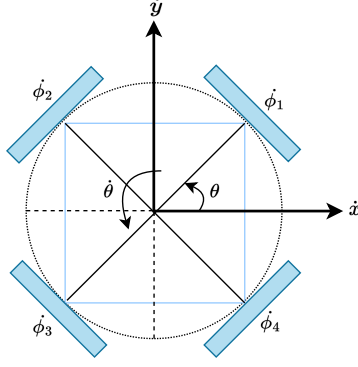


Fig. 2 Kinematic diagram of four-wheel omni robot

The provided model is employed to address both forward and inverse kinematics problems essential for tracking the reference trajectory of the robot, as detailed in the subsequent sections of this paper.

3 Path Planning

We select the A^* algorithm for its efficiency and its appropriateness in devising optimal paths for static environments [3]. This algorithm uses the heuristic approach to estimate the best optimal path by avoiding obstacles in the given 2D environment if the solution really exists. Here, the environment of the robot is discretized into a number of small grid cells called an occupancy grid map filled with the binary digit of '1' and '0' which denotes the obstacle and free space respectively. In Coppeliasim, we have created the custom static obstacles and build the map by treating robot as a point object. So, this increases the map area around the obstacles thereby easing the path planning problem for A^* algorithm. In the following paper [41], different heuristic functions were compared that best optimize the algorithm. But, the simple way is to use Manhattan distance which is based upon the sum of the absolute differences between the two vectors i.e current node to goal node which is given as,

$$ManhattanDistance(c) = |x_c - x_g| + |y_c - y_g| \quad (3)$$

where subscript 'c' is taken as the current node and 'g' is the goal node.

So, the total cost function to be minimized for each node can be calculated as,

$$f(c) = g(c) + h(c) \quad (4)$$

Here, $g(c)$ is the cost value from the starting node to the current node, and $h(c)$ is the heuristic Manhattan function from Equation (3). In order to smooth the path generated by A^* algorithm, we choose to apply B-spline over bezier curve due to its

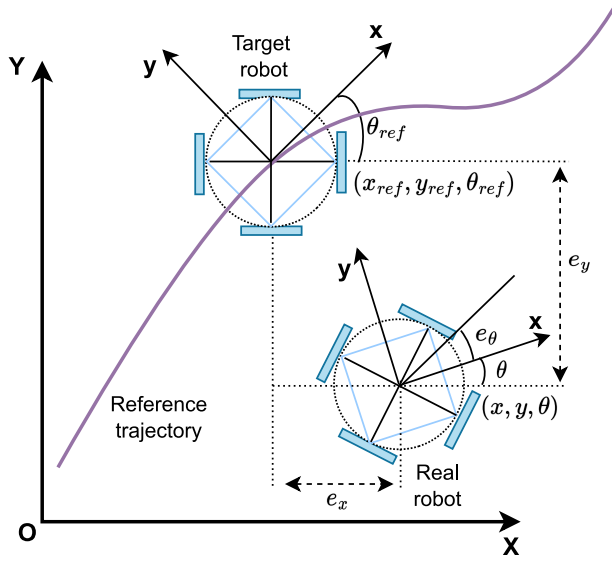


Fig. 3 Trajectory tracking error for Omni-drive robot

superior ability to control local points [4]. Moreover, it exhibits C^2 continuity which is important for stability and passenger comfort.

4 Design of tracking Controller

Given a smooth trajectory generated by B-spline, we obtain N waypoints uniformly sampled at sampling rate of t_s seconds. Here, N is varying depending on the total time defined to reach the destination from the starting point. So, our trajectory waypoints of target robot can be formulated as,

$$\mathbf{X}_{ref}(T) = [\mathbf{x}_{ref}(0), \mathbf{x}_{ref}(t_s), \dots, \mathbf{x}_{ref}(nt_s), \dots, \mathbf{x}_{ref}((N-1)t_s)]^T \quad (5)$$

$$\mathbf{U}_{ref}(T) = [\mathbf{u}_{ref}(0), \mathbf{u}_{ref}(t_s), \dots, \mathbf{u}_{ref}(nt_s), \dots, \mathbf{u}_{ref}((N-1)t_s)]^T \quad (6)$$

where, T is the total specified tracking time, $\mathbf{x}_{ref}(nt_s)$ and $\mathbf{u}_{ref}(nt_s)$ are the intermediate reference waypoint and corresponding velocity respectively. Each waypoint is represented as target robot pose coordinate by $[x_{ref}(nt_s), y_{ref}(nt_s), \theta_{ref}(nt_s)]$ at time nt_s . Here, reference heading angle $\theta_{ref}(nt_s)$ is calculated as,

$$\theta_{ref}(nt_s) = \tan^{-1} \left(\frac{\Delta y_{ref}(nt_s)}{\Delta x_{ref}(nt_s)} \right) \quad (7)$$

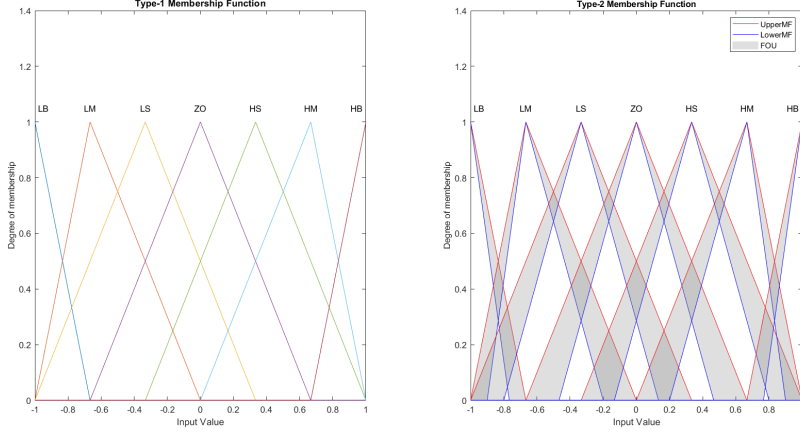


Fig. 4 The range of triangular membership function are adjusted based on input and output variables. Both input variables e and de are in the range between -1 to 1. For output variables (kp, ki, kd), range is adjusted in between -0.1 to 0.1 keeping the structure of membership function similar.

Similarly, we use $[v_{ref}(nt_s), \omega_{ref}(nt_s)]$ to represent reference linear and angular velocity at each sampling time which is given as,

$$\begin{cases} v_{ref}(nt_s) = \frac{\sqrt{(\Delta x_{ref}(nt_s))^2 + (\Delta y_{ref}(nt_s))^2}}{t_s} \\ \omega_{ref}(nt_s) = \frac{\theta_{ref}(nt_s) - \theta_{ref}((n-1)t_s)}{t_s} \end{cases} \quad (8)$$

such that,

$$\begin{cases} \Delta y_{ref}(nt_s) = y_{ref}(nt_s) - y_{ref}((n-1)t_s) \\ \Delta x_{ref}(nt_s) = x_{ref}(nt_s) - x_{ref}((n-1)t_s) \end{cases} \quad (9)$$

In order to track the generated waypoints, the robot position state must also be known in global coordinate system. It is denoted as $\mathbf{X}(T)$ which compose of sequence of pose coordinates of real robot calculated at each sampling interval given by $[x(nt_s), y(nt_s), \theta(nt_s)]$. The whole process can be seen in Fig. 3.

4.1 FuzzyPID controller

The design of the type-1 fuzzy Controller begins with defining input variables $e(k)$ and the change in error $de(k)$. Initially, these are considered as crisp inputs and need to be converted into 7 overlapping fuzzy sets through the process of fuzzification. The universe of discourse for the variables is defined based on the ranges of the input variables. Triangular membership functions are employed for the fuzzification step, where the membership values (degree of truth) are obtained in the range of 0 to 1 on the universe of discourse for the given input values. This process is illustrated in Fig. 4. Fuzzy sets are represented in linguistic form using the set of variables NB,

Table 1 Fuzzy rule for $kp \backslash ki \backslash kd$

$e \backslash de$	NB	NM	NS	ZO	PS	PM	PB
NB	PB\NB\PS	PB\NB\NS	PM\NM\NB	PM\NM\NB	PS\NS\NB	ZO\ZO\NM	ZO\ZO\PS
NM	PB\NB\PS	PB\NB\NS	PM\NM\NB	PS\NS\NM	PS\NS\NM	ZO\ZO\NS	NS\ZO\ZO
NS	PM\NB\ZO	PM\NM\NM	PM\NS\NM	PS\NS\NM	ZO\ZO\NS	NS\PS\NS	NS\PS\ZO
ZO	PM\NM\ZO	PM\NM\NS	PS\NS\NS	ZO\ZO\NS	NS\PS\NS	NM\PM\NS	NM\PM\ZO
PS	PS\NM\ZO	PS\NS\ZO	ZO\ZO\ZO	NS\PS\ZO	NS\PS\ZO	NM\PM\ZO	NM\PB\ZO
PM	PS\ZO\PB	ZO\ZO\NS	NS\PS\PS	NM\PS\PS	NM\PM\PS	NM\PB\PS	NB\PB\PB
PB	ZO\ZO\PB	ZO\ZO\PM	NM\PS\PM	NM\PM\PM	NM\PM\PS	NB\PB\PS	NB\PB\PB

NM, NS, ZO, PS, PM, and PB, where N, Z, H represent Negative, Zero, and Positive respectively, and B, M, S, O represent Big, Medium, Small, and Zero respectively.

The next step involves designing the Mamdani-based fuzzy inference system, where a set of fuzzy rules is defined based on the relationship between fuzzy input sets ($e(k)$ and $de(k)$) and output parameters (kp, ki, kd). These rules are formulated by control engineers based on their experience with various system responses. In this context, we establish 49 rules for each T1-fuzzyPID output parameter, drawing reference from the work in [42]. A fuzzy rule takes the form of an IF(antecedent)-THEN(consequent) statement, such as "if $e(k)$ is PB and $de(k)$ is PB, then apply a large negative kp (i.e., NB)". The process is executed by fuzzy inference engine that employ the max-min composition. The fuzzy rules for each parameter are presented in Table 1.

The final step in the design of the type-1 fuzzyPID controller is defuzzification process, which involves the conversion of linguistic variables into crisp output values using membership functions. This step is the reverse process of fuzzification, where values between 0 and 1 are converted using the Center of Gravity (COG) defuzzification technique. Ultimately, we obtain changes in the PID parameters, which are added to the previous PID parameters to continuously track the reference values, expressed as $K_{pid} = K'_{pid} + \Delta K_{pid}$.

We designed the IT2-Fuzzy PID controller for trajectory tracking in an omnidirectional robot by applying the concepts outlined in Mendel's work on uncertain systems [12]. The key difference between our IT2-FPID and previous controllers lies in the use of Interval Type-2 Fuzzy Sets (IT2 FS), along with type-reduction methods that convert IT2 FS to Type-1 Fuzzy Sets (T1 FS). In this design, IT2 FS are used to represent the fuzzy input and output variables, where uncertainties are captured by the Footprint of Uncertainty (FOU), providing an additional degree of freedom. For our implementation, the error $e(k)$ and its derivative $de(k)$ are represented by seven overlapping FOU, as illustrated in Fig. 4. The upper membership functions (UMFs) of these FOU are identical to the T1 FS, while the scaling factor for the lower membership functions (LMFs) and the lag value are set to 1.0 and 0.3, respectively. Additionally, we adopted the same rule base used in the T1 FPID controller. The firing intervals for each rule are calculated using the minimum implication method. The Karnik-Mendel (KM) algorithms are then employed to obtain T1 FSs by calculating the centroids using the iterative center of sets (COS) type-reduction method. Finally, defuzzification is performed by averaging the left and right points of the type-reduced set to produce the crisp output(PID gains).

Now, tracking the reference trajectory involves calculating the distance and direction errors from robot pose to target reference as,

$$\begin{cases} dr = \sqrt{e_x^2 + e_y^2}, & dr \geq \text{threshold} \\ d\alpha = \tan^{-1} \left(\frac{y_{ref} - y}{x_{ref} - x} \right) - \theta, & -\pi \leq d\alpha \leq \pi \end{cases} \quad (10)$$

such that $e_x = x_{ref} - x$ and $e_y = y_{ref} - y$. The direction error $d\alpha$ forces the robot to follow the target robot with linear velocity v as $\text{fuzzyPID}(dr)$. But, this leads to problem of drifting heading angle which in our case is designed to follow the reference heading angle defined in Equation (7). So, we need addition angular velocity(ω) to do the following task which is calculated as $\text{fuzzyPID}(e_\theta)$ such that $e_\theta = \theta_{ref} - \theta$. Thus, the global velocity of robot is formulated as $[v \cos(d\alpha), v \sin(d\alpha), \omega]$ which is finally passed through the inverse kinematics model to compute individual wheel velocity.

4.2 Non-linear MPC

The Non-linear Model Predictive Controller (NMPC) is an optimization technique that employs the non-linear classical kinematics model to predict the future behavior of a mobile robot within bounded physical constraints on both state variables and control inputs. The cost function is formulated by integrating the tracking error between the predicted states and the true output states within a finite prediction horizon N_p in a sliding mode fashion. Additionally, it includes a term for minimizing the control input error for the target robot velocity. Subsequently, the cost function is processed through a non-linear solver [43], thereby achieving optimal robot states and control sequences for the next N_p prediction horizon. These optimal robot states are closer to the desired path based on the corresponding applied control sequences. Next, we advance one sample time ahead to compute the target direction angle from the first state. Similarly, we consider the first control input velocity, and these steps are repeated for each sampling period with the shifted horizon.

Here, discrete time non-linear model is used to predict the evolution of future states for the mobile robot. Given N_p prediction horizon, following mathematical equation is used to compute state at each sample period $k = nt_s$.

$$\mathbf{x}(k+1 | k) = \mathbf{x}(k | k) + t_s \dot{\mathbf{x}}(k | k) \quad (11)$$

Given that,

$$\begin{aligned} \dot{\mathbf{x}}(k) &= f(\mathbf{x}(k), u(k)) \text{ s.t. } \mathbf{x}(k) \in X, u(k) \in U, \forall k \geq 0 \\ &= [v(k) \cos(\theta(k)), v(k) \sin(\theta(k)), \dot{\theta}(k)]^T \end{aligned} \quad (12)$$

where, $\mathbf{x}(k) \in \mathbb{R}^n$ and $u(k) \in \mathbb{R}^m$ are the state and control vector respectively. Subsequently, we calculate the error between the robot predicted states and reference

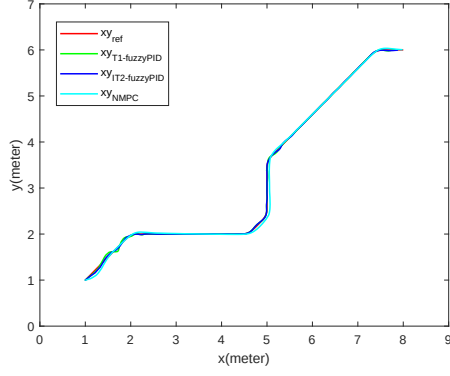


Fig. 5 Comparison of tracking performance of pose(XY) for both type fuzzyPIDs and NMPC with prediction horizon of 15 with a total tracking time of 30 seconds

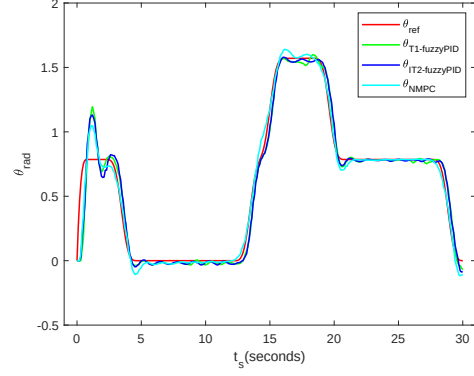


Fig. 6 Comparison of tracking performance of pose(θ) for both type fuzzyPIDs and NMPC with prediction horizon of 15 with a total tracking time of 30 seconds

trajectory as,

$$\begin{cases} \mathbf{e}(k | k) = \bar{\mathbf{x}}(k | k) - \mathbf{x}_{ref}(k | k) \\ \vdots \\ \mathbf{e}(k + N_p | k) = \bar{\mathbf{x}}(k + N_p | k) - \mathbf{x}_{ref}(k + N_p | k) \end{cases} \quad (13)$$

$$\mathbf{x}_{ref}(k | k) = [x_{ref}(k | k) \ y_{ref}(k | k) \ \theta_{ref}(k | k)]^T \quad (14)$$

where, $\bar{\mathbf{x}}(k | k) = \mathbf{x}(k)$ which is robot initial position state. Given target velocity, we also define our objective to minimize the control input error over the finite control horizon of length $N_p - 1$ as defined by equation,

$$\begin{cases} \Delta \mathbf{u}(k | k) = \bar{\mathbf{u}}(k | k) - \mathbf{u}_{ref}(k | k) \\ \vdots \\ \Delta \mathbf{u}(k + i + 1 | k) = \bar{\mathbf{u}}(k + i + 1 | k) - \mathbf{u}_{ref}(k + i + 1 | k) \\ \vdots \\ \Delta \mathbf{u}(k + N_p - 1 | k) = \bar{\mathbf{u}}(k + N_p - 1 | k) - \mathbf{u}_{ref}(k + N_p - 1 | k) \end{cases} \quad (15)$$

$$\mathbf{u}_{ref}(k | k) = [v_{ref}(k | k) \ \omega_{ref}(k | k)]^T \quad (16)$$

Our objective cost function to be optimized is given as,

$$w^* = \min J_w(\mathbf{e}(k | k), \Delta \mathbf{u}(k | k)) = \sum_{k=0}^{N_p} \|\mathbf{e}(k + i | k)\|_{\mathbf{Q}}^2 + \sum_{k=0}^{N_p-1} \|\Delta \mathbf{u}(k + i | k)\|_{\mathbf{R}}^2 \quad (17)$$

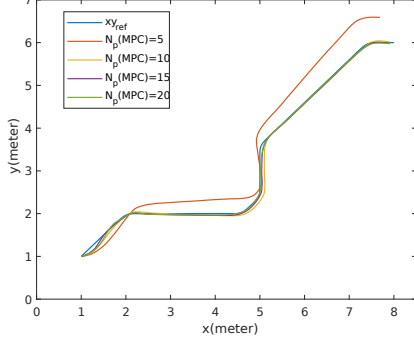


Fig. 7 Comparison of tracking performance of pose(XY) for NMPC with different prediction horizon values with total tracking time of 20 seconds

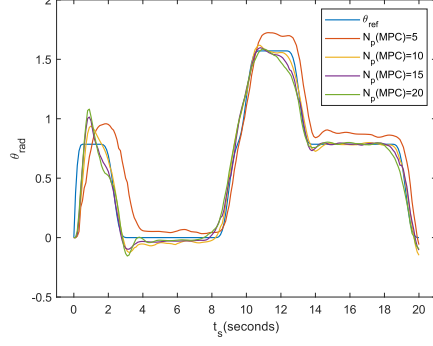


Fig. 8 Comparison of tracking performance of pose(θ) for NMPC with different prediction horizon values with total tracking time of 20 seconds

subject to equality constraints:

$$\mathbf{g}_2(\mathbf{w}) = \begin{bmatrix} \bar{\mathbf{x}}(k) - \mathbf{x}(k) \\ \mathbf{f}(\mathbf{x}(k), \mathbf{u}(k)) - \mathbf{x}(k+1) \\ \vdots \\ \mathbf{f}(\mathbf{x}(k+N_p-1), \mathbf{u}(k+N_p-1)) - \mathbf{x}(k+N_p) \end{bmatrix} = 0 \quad (18)$$

where, $w = [\mathbf{u}_0 \cdots \mathbf{u}_{N_p-1}, \mathbf{x}_0 \cdots \mathbf{x}_{N_p}]$ are problem decision variables which need to be solved. Also the weight matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ and $\mathbf{R} \in \mathbb{R}^{m \times m}$ are tuned for the better accuracy and smooth control inputs. Now, we calculate next output state $\mathbf{x}_1$ based on optimal control input $\mathbf{u}_0$. We extract third element from evolved state which acts as the target direction angle for the Omni-drive robot. Similarly, linear and angular velocity corresponding to first input vector $\mathbf{u}_0$ acts as control inputs which drives the robot to reference pose coordinates.

5 Simulation setup

We conduct trajectory tracking experiments using CoppeliaSim, with our primary objective being the comparison of tracking accuracy between three controllers while keeping the sampling time constant. The tracking performance is simulated over a fixed time intervals of 20 and 30 seconds for the generated trajectory, and the results obtained are plotted for a fair comparison. We also assess the robustness of our proposed controllers in the presence of external noise. The reference and actual 2D-pose values for all the controllers are shown in Fig. 5 and 6. For NMPC, we use a prediction horizon value of 15. It is crucial to note that altering the prediction horizon value has adverse effects on the controller's efficiency, as illustrated in Fig. 7 and 8. The accuracy of the controller is also dependent on the number of intermediate poses

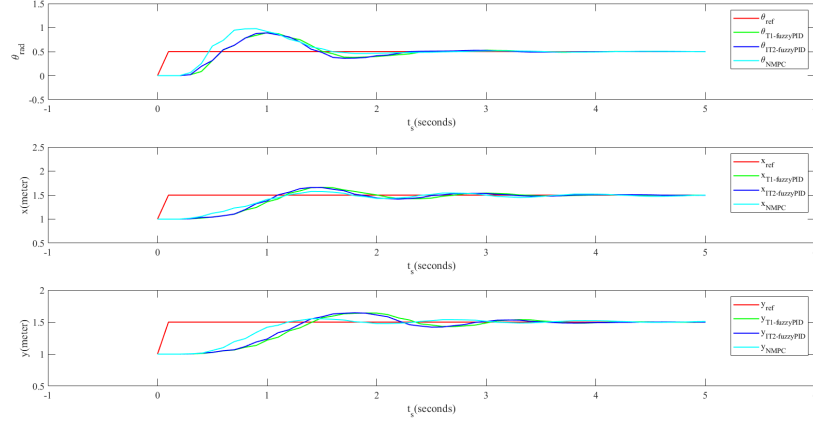


Fig. 9 Step response curve of our controllers

sampled within the given tracking time. In this trajectory experiment, we choose the parameters as, $\mathbf{Q} = 15\mathbb{I}_3$, $\mathbf{R} = \mathbb{I}_2$, $t_s = 0.1s$, $v_{max} = 1.5m/s$, $\omega_{max} = 3.14rad/s$.

We do not take hardware constraints into account as limitations for our simulation platforms. Mean Cross-track Error (ME) and Mean Absolute Error (MAE) metrics serve as the evaluation criteria for judging our results. ME is calculated by averaging the Euclidean distance between the target pose and the real pose at each sampling period. Similarly, we calculate the average value of all absolute deviations between the reference heading angle and the robot heading angle for MAE. Lower error values correspond to better tracking accuracy. Additionally, we also compute the step-response characteristics for each of our designed controllers for more detailed analysis as shown in Fig. 9. On the other hand, we compare the tracking results for different prediction horizon values (N_p) in the case of Non-linear Model Predictive Control.

Table 2 Comparison of tracking error of omni drive robot for both type fuzzyPID and NMPC in absence of noise

Tracking time	T1-fuzzyPID		IT2-fuzzyPID		NPMC($N_p = 15$)	
	ME(XY)	MAE(θ)	ME(XY)	MAE(θ)	ME(XY)	MAE(θ)
20 seconds	0.0874 m	0.0888 rad	0.0855 m	0.0849 rad	0.0722 m	0.0625 rad
30 seconds	0.0521 m	0.05349 rad	0.0501 m	0.0535 rad	0.0439 m	0.0385 rad

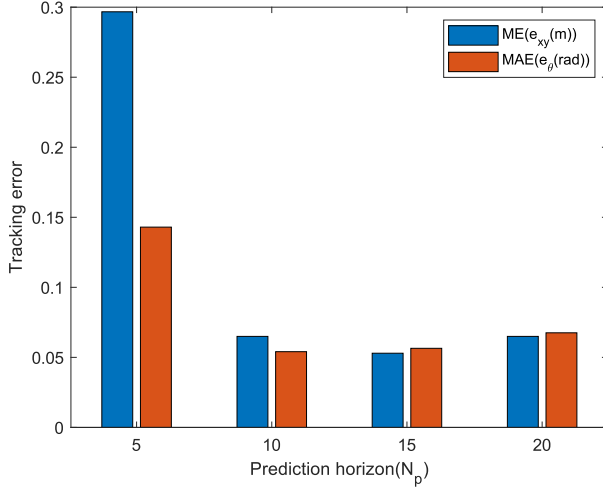


Fig. 10 Comparison of tracking error of robot pose for NMPC with different prediction horizon values with total tracking time of 20 seconds

6 Comparison of fuzzyPIDs and NMPC

6.1 In Absence of Noise

Table 2 presents the tracking error for both fuzzyPIDs and NMPC with a prediction horizon value of 15 in the absence of noise. The NMPC controller achieves a lower error compared to fuzzyPIDs in both metrics while a similar tracking performance is evaluated for fuzzy based controllers. As the tracking time increases, a corresponding improvement in tracking accuracy is observed for both controllers. On the other hand, we found fuzzy Controllers to be more computationally efficient than NMPC, albeit at the expense of tracking accuracy.

Conversely, the performance of NMPC is contingent upon selecting a suitable value for the prediction horizon, resulting in a significant impact on the designed controller's performance. Opting for a lower prediction horizon value leads to suboptimal control actions, rendering the controller less adept at handling the complex dynamics of the system and resulting in lower accuracy. Conversely, a larger prediction horizon increases system complexity and results in a slower response. Furthermore, it introduces a higher risk of overfitting to the prediction model, subsequently yielding sub-optimal results. The error for different values of the prediction horizon is illustrated in Fig. 10.

6.2 Random Noise Injection

To assess the noise rejection capabilities of our proposed controllers for the Omni-drive robot, we introduced randomly generated noise into the feedback path of the input pose state at each timestamp, modeled by the equation $\frac{rand()}{6} \times \sin(\frac{nt_s}{5})$. Simulations

Table 3 Comparison of tracking error of omni drive robot for both type fuzzyPIDs and NMPC in presence of noise

Tracking time	T1-fuzzyPID		IT2-fuzzyPID		NPMC($N_p = 15$)	
	ME(XY)	MAE(θ)	ME(XY)	MAE(θ)	ME(XY)	MAE(θ)
30 seconds	0.0657 m	0.0608 rad	0.0647 m	0.0588 rad	0.0566 m	0.0586 rad

were conducted over 30 seconds, with the resulting trajectory responses displayed in Fig. 11 and Fig. 12. The NMPC controller is found to be more robust to external noise compared to all other controllers. Additionally, the IT2-fuzzyPID controller exhibited greater noise rejection capability than the T1-fuzzyPID controller, attributable to its ability to model uncertainties, despite showing similar tracking performance in the absence of noise. The Mean Error (ME) and Mean Absolute Error (MAE) metrics for the proposed controllers in presence of noise are summarized in Table 3.

Metric	T1-fuzzyPID			IT2-fuzzyPID			NMPCC		
	x	y	θ	x	y	θ	x	y	θ
$M_p\%$	10.800	9.448	78.904	10.611	9.692	77.887	5.083	3.480	95.766
$t_r, 0.1 \rightarrow 0.9$	0.991	1.186	0.215	0.949	1.133	0.242	2.252	0.914	0.175
$t_s \pm 10\%$	1.610	1.186	2.282	1.522	1.133	2.160	0.921	0.914	1.514

Table 4 Step-response characteristics, such as overshoot($M_p\%$), rise time(t_r) and settling time(t_s) of our proposed controllers

6.3 Step Response Analysis

The step-response analysis reveals that the NMPC controller achieves the fastest response, characterized by the minimal overshoot, shortest rise time and settling time, thereby ensuring superior stability. This performance is followed closely by the IT2-fuzzyPID controller, which outperforms the T1-fuzzyPID controller in trajectory tracking. Detailed insights into the step-response characteristics of all controllers are provided in Table 4.

7 Conclusions

In this study, we propose a comprehensive framework for developing a trajectory tracking methodology in a 4-wheel Omni-drive robot. This scheme involves deriving the kinematic model, creating an obstacle-free path, followed by a path-smoothing algorithm. Subsequently, we design two popular control methods, namely fuzzy based PIDs and NPMC, for achieving robust path tracking. The entire setup is simulated, tracking performance is compared by calculating the cross-track error for all of our controllers. Additionally, step-response parameters are computed for clearly highlighting the gap seen in each controllers.

The results indicate that NMPC outperforms fuzzyPIDs, albeit with a trade-off in computational complexity. The NMPC has the better step-response characteristics with low tracking error rate and higher noise rejection capability. Conversely, the

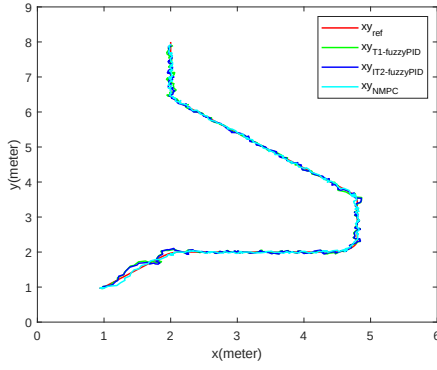


Fig. 11 Comparison of noise rejection capabilities of our controllers on pose(XY)

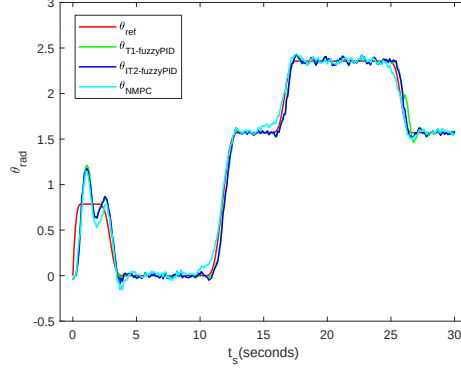


Fig. 12 Comparison of noise rejection capabilities of our controllers on pose(θ)

simplicity of fuzzyPIDs makes it applicable in a wide variety of controller applications. We also assess the effectiveness of NMPC for different values of the prediction horizon in the stability control of our robot.

In a nutshell, the proposed approaches can be useful for a variety of control and robotics tasks, aiming to improve the accuracy and throughput of the system.

8 Limitations and Future Enhancements

While the performance of the aforementioned controllers has been evaluated in simulations, real-time implementation remains untested as they are bound to various external disturbances influencing the stability of mobile robots. Additionally, the simulations were conducted in a static environment limiting their use-case on real world scenarios. Therefore, future work will focus on the real-time implementation of these controllers in both static and dynamic environments to better understand their practical efficacy and adaptability. We also aim to compare the tracking performance of newly proposed IT3 fuzzy system with our proposed approach. Moreover, We also aimed to tune the the sensible gain parameters with the use of various evolutionary algorithms as mentioned in various literature for future work.

References

- [1] Shabalina, K., Sagitov, A., Magid, E.: Comparative analysis of mobile robot wheels design. In: 2018 11th International Conference on Developments in eSystems Engineering (DeSE), pp. 175–179 (2018). IEEE
- [2] Taheri, H., Zhao, C.X.: Omnidirectional mobile robots, mechanisms and navigation approaches. Mechanism and Machine Theory **153**, 103958 (2020)

- [3] Karur, K., Sharma, N., Dharmatti, C., Siegel, J.E.: A survey of path planning algorithms for mobile robots. *Vehicles* **3**(3), 448–468 (2021)
- [4] Ravankar, A., Ravankar, A.A., Kobayashi, Y., Hoshino, Y., Peng, C.-C.: Path smoothing techniques in robot navigation: State-of-the-art, current and future challenges. *Sensors* **18**(9), 3170 (2018)
- [5] Cervantes, I., Alvarez-Ramirez, J.: On the pid tracking control of robot manipulators. *Systems & control letters* **42**(1), 37–46 (2001)
- [6] Somwanshi, D., Bundele, M., Kumar, G., Parashar, G.: Comparison of fuzzy-pid and pid controller for speed control of dc motor using labview. *Procedia Computer Science* **152**, 252–260 (2019)
- [7] Lee, K., Im, D.-Y., Kwak, B., Ryoo, Y.-J., *et al.*: Design of fuzzy-pid controller for path tracking of mobile robot with differential drive. *International Journal of Fuzzy Logic and Intelligent Systems* **18**(3), 220–228 (2018)
- [8] Jiang, W., Jiang, X.: Design of an intelligent temperature control system based on the fuzzy self-tuning pid. *Procedia Engineering* **43**, 307–311 (2012)
- [9] Bansal, U.K., Narvey, R.: Speed control of dc motor using fuzzy pid controller. *Advance in Electronic and Electric Engineering* **3**(9), 1209–1220 (2013)
- [10] Ghanim, T., Ajel, A.R., *et al.*: Optimal fuzzy logic control for temperature control based on social spider optimization. In: *IOP Conference Series: Materials Science and Engineering*, vol. 745, p. 012099 (2020). IOP Publishing
- [11] Chao, C.-T., Sutarna, N., Chiou, J.-S., Wang, C.-J.: An optimal fuzzy pid controller design based on conventional pid control and nonlinear factors. *Applied Sciences* **9**(6), 1224 (2019)
- [12] Mendel, J.M.: Uncertain rule-based fuzzy systems. Introduction and new directions **684** (2017)
- [13] Nour, M., Ooi, J., Chan, K.: Fuzzy logic control vs. conventional pid control of an inverted pendulum robot. In: *2007 International Conference on Intelligent and Advanced Systems*, pp. 209–214 (2007). IEEE
- [14] Alouache, A., Wu, Q.: Fuzzy logic pd controller for trajectory tracking of an autonomous differential drive mobile robot (ie quanser qbot). *Industrial Robot: An International Journal* **45**(1), 23–33 (2018)
- [15] Antonelli, G., Chiaverini, S., Fusco, G.: A fuzzy-logic-based approach for mobile robot path tracking. *IEEE transactions on fuzzy systems* **15**(2), 211–221 (2007)
- [16] Rabah, M., Rohan, A., Han, Y.-J., Kim, S.-H.: Design of fuzzy-pid controller for quadcopter trajectory-tracking. *International Journal of Fuzzy Logic and*

- [17] Bingül, Z., Karahan, O.: A fuzzy logic controller tuned with pso for 2 dof robot trajectory control. *Expert Systems with Applications* **38**(1), 1017–1031 (2011)
- [18] Hashemi, E., Jadidi, M.G., Jadidi, N.G.: Model-based pi-fuzzy control of four-wheeled omni-directional mobile robots. *Robotics and Autonomous Systems* **59**(11), 930–942 (2011)
- [19] Abiyev, R.H., Günsel, I.S., Akkaya, N., Aytac, E., Çağman, A., Abizada, S.: Fuzzy control of omnidirectional robot. *Procedia Computer Science* **120**, 608–616 (2017)
- [20] Masmoudi, M.S., Krichen, N., Masmoudi, M., Derbel, N.: Fuzzy logic controllers design for omnidirectional mobile robot navigation. *Applied soft computing* **49**, 901–919 (2016)
- [21] Mittal, K., Jain, A., Vaisla, K.S., Castillo, O., Kacprzyk, J.: A comprehensive review on type 2 fuzzy logic applications: Past, present and future. *Engineering Applications of Artificial Intelligence* **95**, 103916 (2020)
- [22] Kumbasar, T.: A simple design method for interval type-2 fuzzy pid controllers. *Soft Computing* **18**, 1293–1304 (2014)
- [23] Figueroa, J., Posada, J., Soriano, J., Melgarejo, M., Rojas, S.: A type-2 fuzzy controller for tracking mobile objects in the context of robotic soccer games. In: *The 14th IEEE International Conference on Fuzzy Systems, 2005. FUZZ'05.*, pp. 359–364 (2005). IEEE
- [24] Kumar, A., Kumar, V.: Evolving an interval type-2 fuzzy pid controller for the redundant robotic manipulator. *Expert Systems with Applications* **73**, 161–177 (2017)
- [25] Lu, X., Liu, M.: Optimal design and tuning of pid-type interval type-2 fuzzy logic controllers for delta parallel robots. *International Journal of Advanced Robotic Systems* **13**(3), 96 (2016)
- [26] Humaidi, A.J., Najem, H.T., Al-Dujaili, A.Q., Pereira, D.A., Ibraheem, I.K., Azar, A.T.: Social spider optimization algorithm for tuning parameters in pd-like interval type-2 fuzzy logic controller applied to a parallel robot. *Measurement and Control* **54**(3-4), 303–323 (2021)
- [27] Sarabakha, A., Fu, C., Kayacan, E., Kumbasar, T.: Type-2 fuzzy logic controllers made even simpler: From design to deployment for uavs. *IEEE Transactions on Industrial Electronics* **65**(6), 5069–5077 (2017)
- [28] Vu, T.M., Moezzi, R., Cyrus, J., Hlava, J.: Model predictive control for

- autonomous driving vehicles. *Electronics* **10**(21), 2593 (2021)
- [29] Maurović, I., Baotić, M., Petrović, I.: Explicit model predictive control for trajectory tracking with mobile robots. In: 2011 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), pp. 712–717 (2011). IEEE
 - [30] Liu, X., Wang, W., Li, X., Liu, F., He, Z., Yao, Y., Ruan, H., Zhang, T.: Mpc-based high-speed trajectory tracking for 4wis robot. *ISA transactions* **123**, 413–424 (2022)
 - [31] Kanjanawanishkul, K., Zell, A.: Path following for an omnidirectional mobile robot based on model predictive control. In: 2009 IEEE International Conference on Robotics and Automation, pp. 3341–3346 (2009). IEEE
 - [32] Yang, H., Guo, M., Xia, Y., Cheng, L.: Trajectory tracking for wheeled mobile robots via model predictive control with softening constraints. *IET Control Theory & Applications* **12**(2), 206–214 (2018)
 - [33] Nascimento, T.P., Dórea, C.E.T., Gonçalves, L.M.G.: Nonlinear model predictive control for trajectory tracking of nonholonomic mobile robots: A modified approach. *International Journal of Advanced Robotic Systems* **15**(1), 1729881418760461 (2018)
 - [34] Pacheco, L., Luo, N.: Testing pid and mpc performance for mobile robot local path-following. *International Journal of Advanced Robotic Systems* **12**(11), 155 (2015)
 - [35] Mayne, D.Q.: Model predictive control: Recent developments and future promise. *Automatica* **50**(12), 2967–2986 (2014)
 - [36] Bai, G., Meng, Y., Liu, L., Luo, W., Gu, Q., Liu, L.: Review and comparison of path tracking based on model predictive control. *Electronics* **8**(10), 1077 (2019)
 - [37] Conceição, A.S., Oliveira, H.P., Silva, A.S., Oliveira, D., Moreira, A.P.: A nonlinear model predictive control of an omni-directional mobile robot. In: 2007 IEEE International Symposium on Industrial Electronics, pp. 2161–2166 (2007). IEEE
 - [38] Wang, C., Liu, X., Yang, X., Hu, F., Jiang, A., Yang, C.: Trajectory tracking of an omni-directional wheeled mobile robot using a model predictive control strategy. *Applied Sciences* **8**(2), 231 (2018)
 - [39] Grüne, L., Pannek, J., Grüne, L., Pannek, J.: Nonlinear model predictive control. *Nonlinear Model Predictive Control: Theory and Algorithms*, 43–66 (2011)
 - [40] Baede, T.: Motion Control of an Omnidirectional Mobile Robot
 - [41] Jing, X., Yang, X.: Application and improvement of heuristic function in a* algorithm. In: 2018 37th Chinese Control Conference (CCC), pp. 2191–2194 (2018).

IEEE

- [42] Xu, X., Wang, Q.: Speed control of hydraulic elevator by using pid controller and self-tuning fuzzy pid controller. In: 2017 32nd Youth Academic Annual Conference of Chinese Association of Automation (YAC), pp. 812–817 (2017). IEEE
- [43] Andersson, J.A., Gillis, J., Horn, G., Rawlings, J.B., Diehl, M.: Casadi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation* **11**, 1–36 (2019)